

SOLANA VALUATION INTERFACE

Pricing that anyone can recompute.

The value of a protocol asset, computed on-chain from accounts anyone can verify and written to an account anyone can read — **so nobody has to trust a server doing hidden math.**

First feed live on a mainnet fork: Hylo xSOL NAV

Prince · SVI · September 2026

It works. Here is the receipt.

10 September 2026. A local fork of Solana mainnet, Hylo's real accounts re-cloned seconds before. One transaction, ~13,000 compute units.

\$0.0603

per xSOL — \$0.060285498,
computed on-chain by hylo-
core

4.9 bps

wide band: redeem
\$0.060285498,
mint \$0.060315053

0

difference from an independent
off-chain recomputation

~13k CU

to refresh; ~200 CU
for a consumer to read

PUBLISHED QUOTE · G28Ba5F9X71Pih2SNobXyU98tjQdga1Ztbzqg3PU4rWj

```
quote_amount    $ 0.060285498
lower / upper   $ 0.060285498 .. $ 0.060315053
base_amount     1000000 (1 whole xSOL)
observed_slot   445953445   valid_until 445954195
sequence        1           flags    BUY_ZONE
off-chain hylo-core: $ 0.060285498   MATCH
```

What this proves

- The adapter read Hylo's real state and priced xSOL with Hylo's own library
- It called svi-core over a frozen byte format, signed by its own PDA
- svi-core wrote the quote; a consumer read it back by byte offset
- An off-chain reader over the same bytes agreed to the last digit

“

honestly an accepted smart contract interface for pricing onchain assets would be amazing — bc pyth wants us to read our own protocol by RPC and give them data by an API — it leaves all sorts of unimaginable security holes

© Plish, Hylo founder · public post, 28 August 2026

Nobody publishes the number.

xSOL's value is a formula over on-chain state. No oracle carries it; Hylo's site computes it in your browser.

The workaround is the hole.

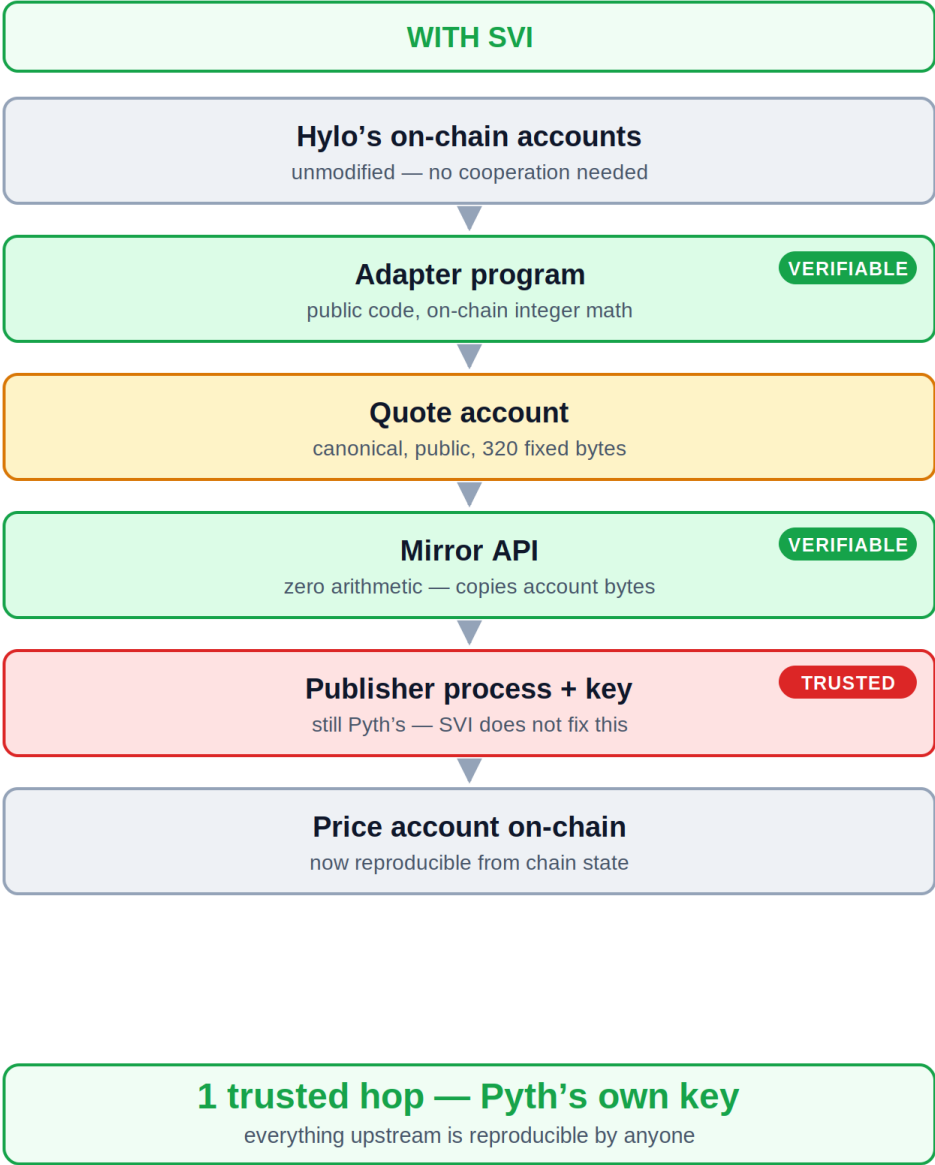
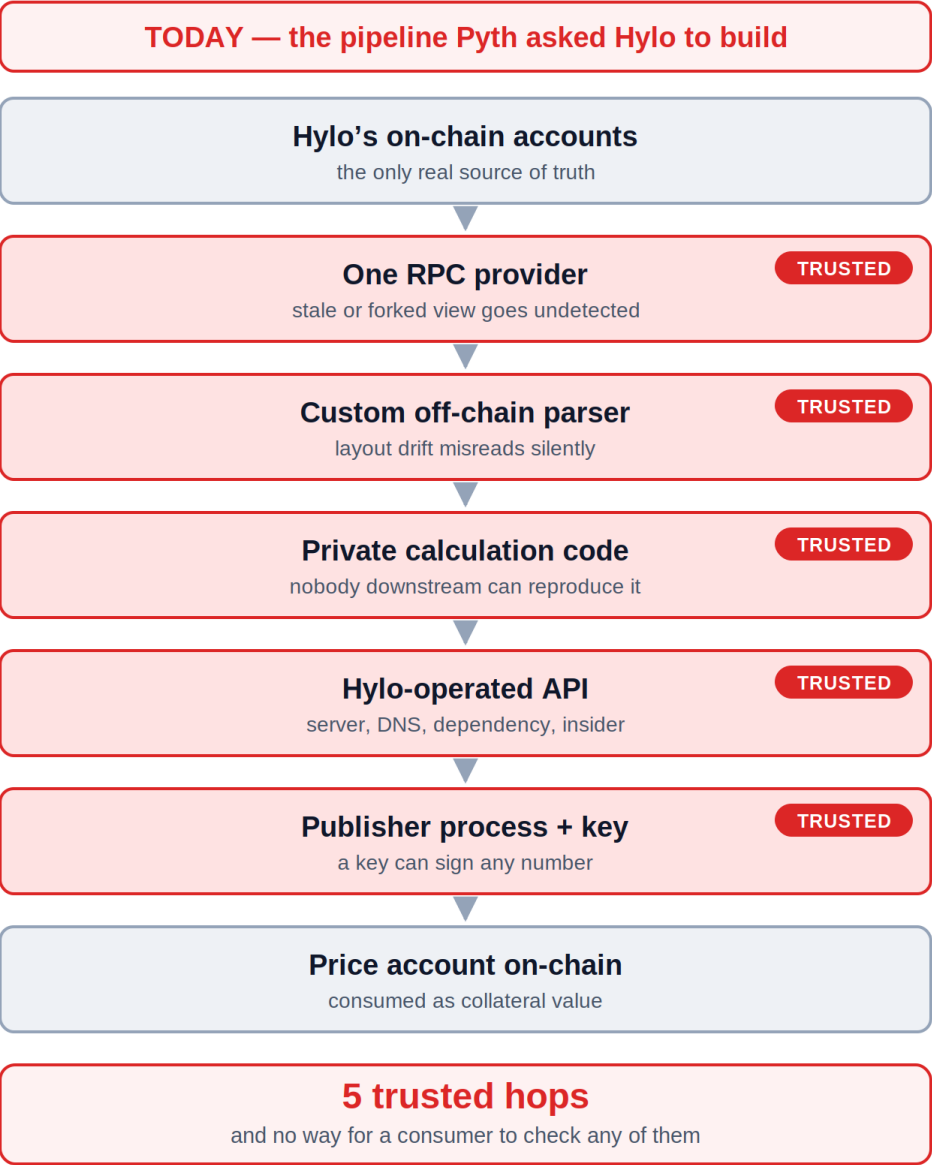
Read your own protocol by RPC, do the math on a server, hand a number to an API. Five trusted hops; a consumer can check none.

It is not hypothetical.

Four lenders have already lost money to exactly this failure class. The next slide has the bill.

Where does the number come from?

Five places the value can change without anyone noticing — versus one, whose output anyone can reproduce from the same slot’s account data.



This failure class already has a body count

In every case below, no on-chain account was ever wrong. The loss came entirely from the valuation layer between true state and the consumer.

\$5.8M

Loopscale

Apr 2025 · Solana

The lender's own pricing of RateX PT tokens was wrong. Undercollateralised loans drained 12% of TVL — two weeks after launch.

\$1.8M

Moonwell

2025 · bad debt

A config change priced cbETH via cbETH/ETH only, dropping the ETH/USD leg. It read \$1.12 instead of ~\$2,200. Bots did the rest.

1.06 → 1.7

wUSDM / Venus

Feb 2026

A donation attack moved the ERC-4626 share exchange rate itself, then self-liquidated against it. The rate was the attack surface.

\$10.2M

YieldBlox

Feb 2026

Thin-liquidity VWAP oracle on a tokenised-treasury pair inflated collateral value enough to over-borrow.

\$137M+

lost across DeFi exploits in Q1 2026 alone; over \$1B cumulatively in 2026.

Loopscale lists hyUSD today. A lender that hand-rolls valuation for every protocol asset it accepts will eventually hand-roll one wrong.

THE IDEA

Move the arithmetic on-chain.

Then RPCs, APIs and keys become couriers —
not authors — of the number.

Computed

by a Solana program, from accounts it verifies itself, with the protocol's own math library

Published

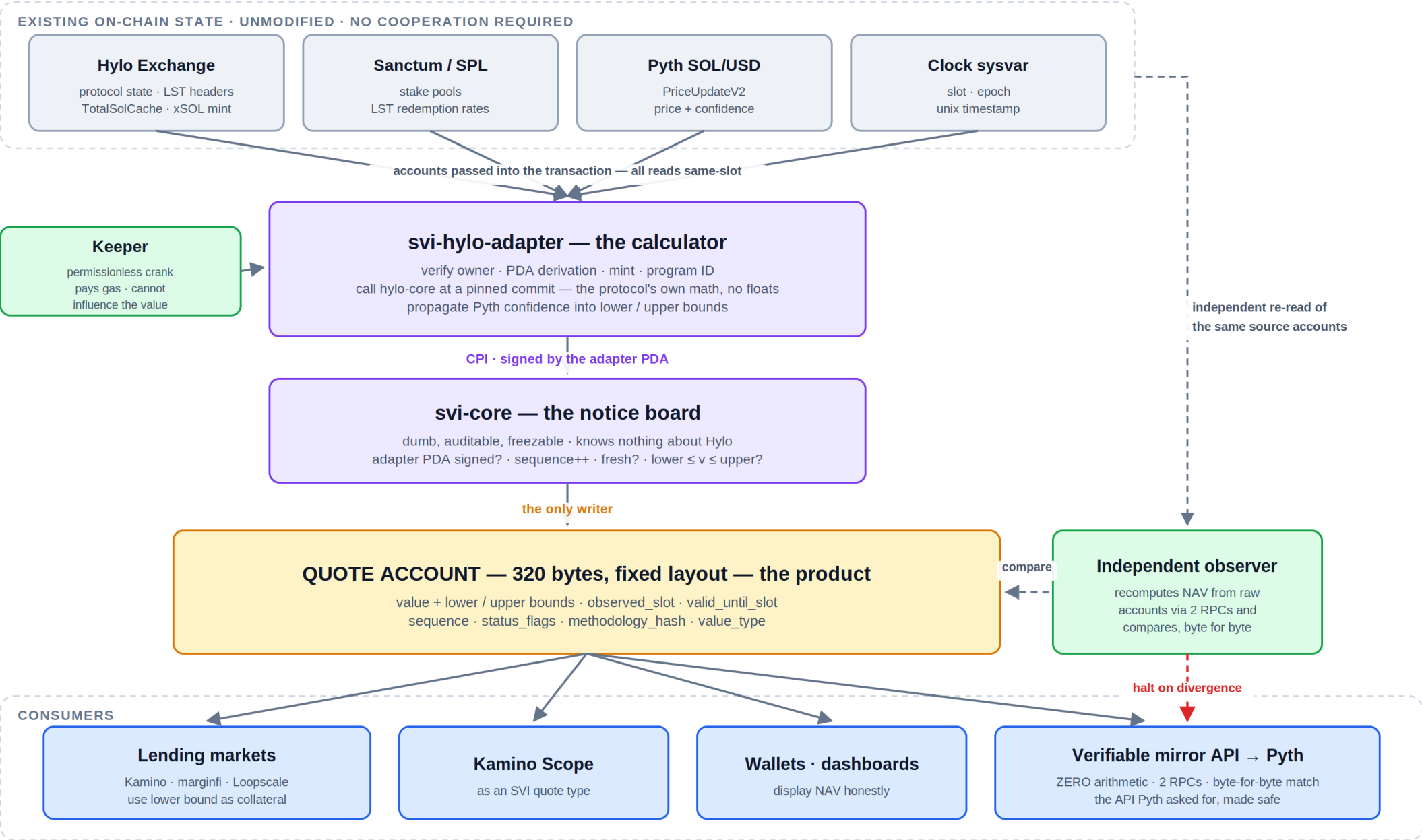
to a 320-byte account with a fixed layout that anyone can read for ~200 CU

Reproducible

byte-for-byte by any third party over the same accounts at the same slot

Solana Valuation Interface — system architecture

The value is computed on-chain from verified accounts and written to a public account. No off-chain service is ever trusted with the arithmetic.



Fail stale, never fail wrong. Any unverified input aborts the refresh — the quote ages out and consumers see the expiry. No code path publishes a value computed from unvalidated inputs.

Do we depend on Pyth, or does Pyth depend on us?

Today: we depend on Pyth

Pyth answers one question — what is 1 SOL worth right now — from exchanges. xSOL's value is calculated from that answer plus Hylo's books: collateral $\times$ SOL price, minus hyUSD debt, divided by supply. Pyth is our input. Nobody else can be.

So our quote can never be fresher than the SOL price it came from. Hylo's own rule is that a price older than 10 seconds is unusable; the adapter inherits that rule unchanged and refuses rather than publish from a stale input.

Later: Pyth could depend on us

Nobody publishes the output — xSOL's NAV — on-chain. Once SVI does, an oracle network can consume that account to offer an xSOL/USD feed, the way it wraps other on-chain sources, instead of asking each protocol to run a server and an API.

That is the request in Plish's post, answered without the security hole: the oracle reads a public account whose contents anyone can recompute.

The claim the demo supports

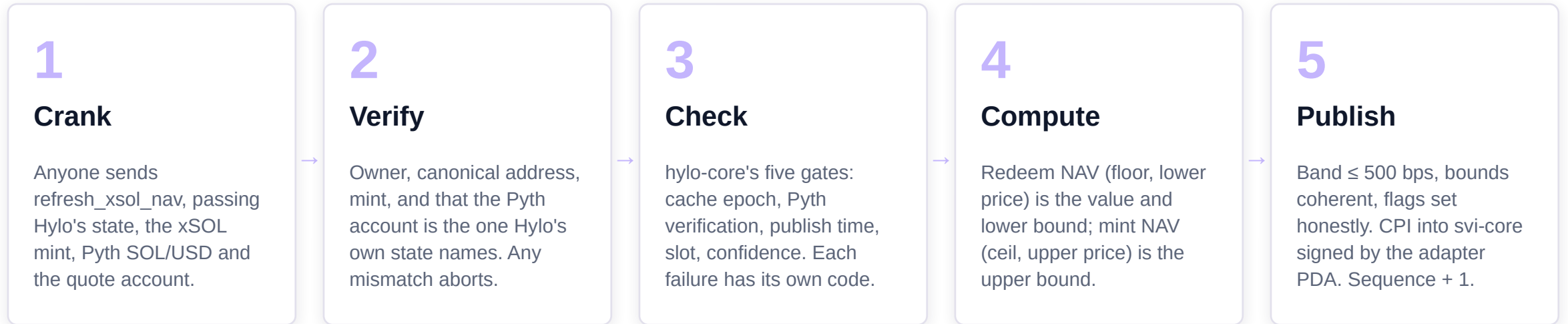
Not **“fresher than Pyth”**. It is: **the number nobody publishes, now on-chain, moving 2.5 $\times$ SOL, with bounds and a staleness horizon, written only by code.**

Five pieces, and what each one is for

- | | | |
|---|--|---|
| 1 | Adapter <i>the calculator</i> | One per protocol. Reads the protocol's public account bytes, verifies owner / PDA / mint / oracle, and calls the protocol's own math library. Reimplements nothing. |
| 2 | Core <i>the notice board</i> | Tiny, protocol-agnostic, no arithmetic. Only the adapter PDA named in the feed descriptor may write. Sequence monotonic, validity stamped, bounds checked. |
| 3 | Quote account <i>the product</i> | 320 fixed bytes: value, lower/upper bounds, observed slot, valid_until, sequence, status flags, methodology hash. Read for ~200 CU. |
| 4 | Keeper <i>rings the bell</i> | Permissionless — anyone can crank. Pays gas; cannot influence the value. If nobody cranks, the quote expires visibly. Built; ships with a watcher. |
| 5 | Observer <i>the second accountant</i> | Recomputes from raw accounts through independent RPCs. A divergence of one unit alerts and halts the mirror. Next. |

The principle it all reduces to: there is no point in the pipeline where a human or a hackable server chooses the number.

One transaction, every read from the same slot



Permissionless, yet uninfluenceable

Anyone can send this transaction. Nobody can change what it writes — the value is computed by adapter code from verified accounts, and only the adapter's PDA can authorise the write. The keeper is paying for gas. If our bot dies, anyone else's bot cranks it.

We reimplement none of Hylo's math

The adapter links hylo-core — the library Hylo's own program uses — pinned to one commit, recorded on-chain in the adapter config. `levercoin_redeem_nav` and `levercoin_mint_nav` are called, not copied. Hylo's redemption math and SVI's published value cannot diverge, because they are the same code.

THE DESIGN RULE

Fail stale, never fail wrong.

No code path may publish a value computed from unvalidated inputs. A visibly expired quote is always preferable to a plausible wrong one.

Condition	Behaviour	What the consumer sees
Everything healthy	Publish, sequence + 1	Fresh quote, flags clear
Hylo's collateral cache is from a past epoch	Abort — no write (HyloCacheStale)	Quote ages past valid_until_slot
Pyth price older than Hylo's 10 s window	Abort — no write (OracleStale)	Quote ages past valid_until_slot
Pyth confidence wider than Hylo tolerates	Abort — no write (OracleConfidenceTooWide)	Quote ages past valid_until_slot
Mint/redeem band over 500 bps	Abort — no write (BandTooWide)	Quote ages past valid_until_slot
Hylo underwater (collateral ratio < 100%)	Publish exactly 0 with halt flags	DESTABILIZED — collateral value zero, not a dip to buy
xSOL supply is 0	Publish 1.000000000, flagged	ZERO_SUPPLY_DEFAULT
Wrong Pyth account passed by a keeper	Abort — no write (OracleMismatch)	Nothing; the keeper was wrong or malicious

Every row above is implemented and tested. SVI imposes no oracle tolerance of its own — it reads Hylo's on-chain OracleConfig on every refresh, so the feed reflects exactly what the protocol would accept for a real redemption.

Three ways to consume it, one of them dominant

A Read the account 99% of usage

- Pass the quote account into your own instruction
- Check `valid_until_slot` and `status_flags`
- Check `value_type` is what you onboarded
- Use the lower bound for collateral

One account read, ~200 CU. Fixed 320-byte layout, asserted by a test.

B Call the calculator live high-stakes only

- Include the adapter's compute in your own transaction
- Value derived from this exact slot's state, atomically
- More accounts, ~13k compute units
- Zero staleness

For liquidators who cannot accept a five-minute-old quote. Most integrators never need this.

C The verified mirror off-chain consumers

- Fetch the account through two independent RPCs
- Require byte-for-byte agreement
- Decode and serve JSON with full provenance
- Serve an error when stale — never a last-known-good value

A photocopier. It cannot lie about the price, because anyone can compare its output to the account. Not built yet.

A worked example: 100,000 xSOL deposited → the market reads `valid_until` ≥ now, flags clear, lower bound \$0.060285498 → \$6,028.55 of collateral → \$3,617 borrowable at 60% LTV. When `valid_until` passes, refuse new borrows; never guess.

What exists, and what is next

Component	State	What it is
programs/svi-core	DONE	Descriptor + 320-byte quote, layout asserted by test. Accepts a write only from the adapter PDA named in the descriptor. Sequence, validity, bounds enforced.
crates/svi-abi	DONE	The 96-byte wire format between adapter and core. Zero dependencies, so programs on incompatible framework versions share it safely. Checked from both sides.
programs/svi-hylo-adapter	DONE	Verifies accounts, links hylo-core at a pinned commit, publishes redeem/mint NAV with flags. 17 tests, including a live end-to-end run on a mainnet fork.
First on-chain publication	PROVEN	10 Sep 2026: \$0.060285498, band 4.9 bps, exact match with an independent off-chain recomputation. Recorded in docs/validation.
tools/svi-keeper	DONE	Permissionless crank with decoded refusal codes, plus a watcher for the quote and for the Pyth input.
hylo-xsol-nav-v1 spec	DRAFT	Complete except the five open questions in §10. Frozen and hashed before mainnet.
Mainnet · observer · mirror · audit	NEXT	Mainnet deploy $\approx$ 3.3 SOL; keeper $\approx$ 0.1 SOL/day. Observer and mirror after. No real collateral against these feeds until a third-party audit.

The critical path is no longer engineering. It is a frozen spec, a mainnet deployment, and a first consumer.

The asset class is NAV-shaped, and it is growing fast

\$3.7B

tokenised RWA on Solana,
up 4× in six months

~\$15B

Solana LST supply —
every unit a formula, not a
trade

~\$3.5B

lending TVL carrying this
valuation risk today

2,800+

Pyth feeds — built for
observed market prices

Why now

- A protocol founder asked for exactly this, in public, in August — because Pyth is asking him for an API.
- Hylo's next version multiplies 4 feeds into $4 \times N$ assets.
- Solana RWA quadrupled in six months, and it is NAV-priced by definition.
- The Foundation is standardising the neighbouring layer (sRFC 40) right now.
- Colosseum runs 28 Sep – 2 Nov 2026. The first feed already works.

The honest version

Standards do not monetise. Realistic year-one revenue is low six figures at best, from services beside a permanently free standard — custom adapters, managed keepers, reviews. There is no token and there will not be one.

The return is position: authoring the valuation interface Solana adopts, with live feeds, while the Foundation standardises next door. Cost to find out: about 3.3 SOL to deploy and \$50–150 a month to run.

Nobody does this as an open interface

Player	What it is	Why SVI is not redundant
Pyth	Push/pull oracle, 2,800+ feeds, 128 publishers	Upstream input and prospective consumer, not competitor. Built for observed market prices; it has no mechanism for reproducible protocol NAV, which is why it asks protocols for APIs.
Switchboard	Permissionless custom feeds, TEE-executed jobs	Real overlap, but the math still runs off-chain in an enclave. You trade 'trust the API' for 'trust the enclave and the job definition'.
Kamino Scope	Oracle aggregator, ~40 asset-specific valuation types	Closest analogue and proof the adapter model works — but it is Kamino's internal component, curated by and for Kamino. A consumer target, not a rival.
Blueshift Doppler	Ultra-efficient price publishing primitive	Transport, not semantics. It makes writing a number cheap; it says nothing about where the number came from.
sRFC 40	Foundation + Exo vault standard, NAV-settled flows	The one genuine strategic risk. Response is to contribute into it, not compete — and to cover what vaults don't: leveraged tokens, backing NAV, redemption, receipt tokens.
Chainlink NAV	Brings off-chain fund NAV on-chain for RWAs	Opposite direction: it transports NAV computed off-chain by a fund administrator. SVI computes NAV from on-chain state.

The honest read: nobody does exactly this, and not because it is hard — because it does not obviously monetise. Kamino, Switchboard and Blueshift could each build it in weeks. The defensible assets are methodology specs, audits, track record and integrations — not code. Which is why the code is open.

Six stages, each with a gate before the next

✓	Prove it on a fork	Adapter publishes real xSOL NAV; off-chain recomputation matches exactly.	DONE · 10 Sep
1	Mainnet + keeper	Spec frozen and hashed. Both programs deployed. Keeper running; watcher public.	NOW · ~2 weeks
2	Observer + audit	Zero divergence over 72 h through two RPCs. Third-party audit before any real collateral.	Oct 2026
3	First consumer	One lending market or dashboard reads the quote account in production.	Q4 2026
4	Adapter #2	A second protocol ships with zero changes to core or the wire format.	Q1 2027
5	Into the sRFC	Submitted aligned with sRFC 40, not parallel to it.	Q1–Q2 2027

Stages 0–2 are within our control and only prove competence. Stages 3 and 4 are the real test — until a second protocol adopts it, SVI is a Hylo wrapper, and we say so.

Three things, from three audiences

SUPERTEAM

An instagrant

Covers the audit, mainnet deployment, and three months of keeper and observer operation. Everything it funds is open source and permanently free.

LENDERS & DASHBOARDS

One integration

Read one 320-byte account. Kamino, marginfi, Loopscale, or any wallet that shows xSOL. A pilot on devnet first; mainnet after the audit.

HYLO

Fifteen minutes

Read §§2–6 of the spec and tell us if we misread your math. No program changes, no key, no engineering time. The adapter runs whether or not you answer.

NOT ASKED FOR

A token · Exclusivity · Hylo's engineering time or permission · Anyone to trust our server — there is no server in the loop

Colosseum runs 28 Sep – 2 Nov 2026. A mainnet-live feed with a keeper and an observer is the submission.

The objections worth raising, answered

“Isn't this just a Hylo wrapper?”

Today, yes, and the roadmap says so. The interface is the product: a second adapter that ships with zero changes to core or the wire format is the test, and it is stage 4. Until then, judge the design by whether the first adapter needed any cooperation from Hylo. It did not.

“What if the adapter is wrong and someone gets liquidated?”

A real risk. Hence: call hylo-core rather than reimplement it; refuse rather than publish on any unverified input; an observer that halts on a one-unit divergence; and no real collateral until a third-party audit. Until that audit, our own position is that no market should underwrite against these feeds.

“Is this a hackathon project that dies in November?”

The mitigations hold regardless: everything open source, the keeper permissionless, the core small enough to freeze, and the methodology spec standing on its own as documentation of how xSOL is valued. If every line of SVI code were abandoned, the quote account keeps working until nobody cranks it — visibly.

“What's in it for you?”

Reputation, a Colosseum submission with a working feed, and a shot at authoring a Solana standard. Not revenue — the standard and SDKs are permanently free and there is no token. Worth being blunt about, because an unexplained motivation is its own kind of risk.

WHAT HAPPENS NEXT

Week 1	Freeze hylo-xsol-nav-v1 at v1.0; publish the hash; deploy both programs to mainnet.
Week 2	Keeper live. Quote account address published. Watcher public so anyone can see it tick.
Week 3–4	Observer at zero divergence over 72 hours. Audit scoped.
October	Colosseum submission: a live feed, a keeper, an observer, and a spec.
Then	First consumer on devnet. Second adapter. Into the sRFC.

Everything in this deck is in the repository: the programs, the tests, the keeper, the spec, the validation record with the transaction signature, and the page that explains how the number is made.

Fail stale, never fail wrong.

github.com/princdotrs/solana-valuation-interface

Prince · @princdotrs